

Striver's A2Z DSA Sheet

Your Ultimate Systematic Roadmap from Absolute Beginner to Advanced Interview Readiness

Target: MAANG & Top Tech Product Companies

Structure: 18 Steps | 450+ Curated Problems

Status Tracker: ☐ Structured Checklists

Step 1: Learn the Basics

1.1 Things to Know in C++/Java/Python or any language

DONE	PROBLEM / TOPIC NAME	DIFFICULTY	KEY CONCEPT FOCUS
☐	User Input / Output & Data Types	Easy	I/O streams, primitive size bounds
☐	If Else statements & Switch Case	Easy	Conditional flows, execution branches
☐	For Loops, While Loops, Do-While Loops	Easy	Iterators, entry/exit criteria
☐	Functions (Pass by Value and Reference)	Easy	Memory stack layout, pointers/aliases

1.2 Build up Logical Thinking (Pattern Solving)

DONE	PROBLEM / TOPIC NAME	DIFFICULTY	KEY CONCEPT FOCUS
☐	Patterns 1 to 11 (Square, Triangles, Inverted Pyramids)	Easy	Nested loop structural control
☐	Patterns 12 to 22 (Number symmetry, Diamonds, Borders)	Easy	Index mapping formulas

1.3 Complete C++ STL or Java Collections Framework

DONE	PROBLEM / TOPIC NAME	DIFFICULTY	KEY CONCEPT FOCUS
☐	Vectors, Lists, Deque, Stack, Queues	Easy	Sequence containers & iterators
☐	Set, Multiset, Map, Unordered Map	Easy	Balanced trees vs Hashing interfaces

Step 2: Learn Important Sorting Techniques

DONE	PROBLEM / TOPIC NAME	DIFFICULTY	TIME COMPLEXITY
☐	Selection Sort & Bubble Sort	Easy	$O(N^2)$ worst & average
☐	Insertion Sort	Easy	$O(N)$ best case adaptive
☐	Merge Sort (Divide & Conquer)	Medium	$O(N \log N)$ stable runtime
☐	Quick Sort (Pivot Partitioning)	Medium	$O(N \log N)$ average, in-place

Step 3: Solve Problems on Arrays [Easy -> Medium -> Hard]

3.1 Arrays - Easy Level

DONE	PROBLEM NAME	DIFFICULTY	OPTIMAL APPROACH
☐	Largest & Second Largest Element in an Array	Easy	Single pass linear scan
☐	Check if Array is Sorted and Rotated	Easy	Count inversion transitions (≤ 1)
☐	Remove Duplicates from Sorted Array	Easy	Two-pointer unique placement
☐	Left Rotate an Array by K places	Easy	Block reversal algorithm
☐	Move Zeroes to End	Easy	Two-pointer active swap
☐	Find the Missing Number & Max Consecutive Ones	Easy	Sum formula / XOR accumulator

3.2 Arrays - Medium Level

DONE	PROBLEM NAME	DIFFICULTY	CLASSIC ALGORITHM
☐	Two Sum Problem	Medium	Hash Map lookup / Sorting
☐	Sort an array of 0s, 1s and 2s	Medium	Dutch National Flag Algorithm
☐	Majority Element ($> N/2$ times)	Medium	Boyer-Moore Voting Algorithm
☐	Maximum Subarray Sum	Medium	Kadane's Algorithm ★
☐	Stock Buy and Sell	Easy	Tracking running minimal valley
☐	Next Permutation	Medium	Lexicographical breakpoint search
☐	Leaders in an Array	Easy	Scan from right-to-left tracking max
☐	Longest Consecutive Sequence	Medium	HashSet lookup sequence heads
☐	Rotate Matrix by 90 Degrees	Medium	Transpose matrix + Reverse rows

3.3 Arrays - Hard Level

DONE	PROBLEM NAME	DIFFICULTY	ADVANCED MECHANICS
☐	3-Sum and 4-Sum Problems	Hard	Sorting + Nested Fixed Pointers
☐	Merge Overlapping Subintervals	Medium	Sorting by start intervals
☐	Merge Two Sorted Arrays Without Extra Space	Hard	Gap Method / Insertion variants
☐	Find Missing and Repeating Number	Hard	Math equations / Bitwise XOR
☐	Inversion Count of an Array	Hard	Modified Merge Sort merge-step
☐	Maximum Product Subarray	Hard	Prefix & Suffix dual-tracked scan

Step 4: Binary Search [1D, 2D Arrays & Search Space]

4.1 BS on 1D Arrays

DONE	PROBLEM NAME	DIFFICULTY	LOGIC FOCUS
☐	Binary Search Target Find	Easy	Divide space bounds by half
☐	Implement Lower Bound / Upper Bound	Easy	First index verifying condition
☐	Search in Rotated Sorted Array (I & II)	Medium	Identify natively sorted zone
☐	Find Peak Element	Medium	Local slope property binary check

4.2 BS on Answers (Minimizing Max / Maximizing Min)

DONE	PROBLEM NAME	DIFFICULTY	FEASIBILITY CHECK
☐	Square Root of an integer / Nth Root	Easy	Range bounds [1, N] search
☐	Koko Eating Bananas	Medium	Sum of ceiling divisions $\leq H$
☐	Allocate Books / Split Array Largest Sum	Hard	Greedy contiguous allocation test
☐	Aggressive Cows	Hard	Maximize minimal placement delta

Step 5: Strings [Easy & Medium/Hard]

DONE	PROBLEM NAME	DIFFICULTY	KEY STRATEGY
☐	Reverse Words in a String	Easy	Tokenize parsing or clean swap
☐	Largest Odd Number in String	Easy	Scan backward to locate odd digit
☐	Longest Common Prefix	Easy	Horizontal or vertical scan match
☐	Valid Anagram & Isomorphic Strings	Easy	Frequency mapping hash array
☐	Longest Palindromic Substring	Medium	Expand from center / DP approach
☐	Roman to Integer / Integer to Roman	Medium	Lookahead value evaluation subtraction

Step 6: Linked List [Singly, Doubly, Loop Detection, Logic]

6.1 Core Structural Foundations

DONE	PROBLEM NAME	DIFFICULTY	KEY POINTER WORK
☐	Introduction to Singly & Doubly LL	Easy	Node allocation & link bindings
☐	Insert/Delete Node at Position K	Easy	Pointer reconfiguration logic
☐	Reverse a Linked List (Iterative & Recursive)	Medium	Three-pointer swap cycle

6.2 Advanced Loop & structural properties

DONE	PROBLEM NAME	DIFFICULTY	CLASSIC STRATEGY
☐	Detect Cycle in a Linked List	Easy	Floyd's Tortoise and Hare
☐	Find Starting Node of the Loop	Medium	Reset slow pointer to head + phase 2
☐	Find Intersection Node of Two Lists	Medium	Dual pointer differential cycle match
☐	Delete Middle Node / Find Middle Node	Easy	Fast and slow reference skips
☐	Flattening a Linked List	Hard	Priority queue merge vertical heads

Step 7: Recursion & Backtracking

DONE	PROBLEM NAME	DIFFICULTY	ALGORITHMIC LOGIC
☐	Generate all Subsequences / Subsets	Medium	Pick / Don't Pick recursion tree
☐	Combination Sum (I & II)	Medium	Backtracking with sort to avoid dupes
☐	Subset Sums I & II	Medium	DFS combinatorial generation
☐	N-Queens Problem	Hard	Column & Diagonal collision safety
☐	Sudoku Solver	Hard	Exhaustive board cell value trials
☐	Rat in a Maze	Hard	Visited matrix backtracking paths

Step 8: Bit Manipulation

DONE	PROBLEM NAME	DIFFICULTY	BITWISE TRICK
☐	Check if i-th bit is set or not	Easy	Bitwise AND with $(1 \ll i)$
☐	Set/Clear/Toggle the i-th bit	Easy	Use OR ($ $), AND ($\sim$), XOR ($\wedge$) operators
☐	Check if a number is Power of 2	Easy	Expression: $(n \& (n - 1)) == 0$
☐	Count Number of Set Bits (Hamming Weight)	Easy	Brian Kernighan's bit clearance loop
☐	Two Single Numbers (Others appear twice)	Medium	XOR bucket split using lowest set bit

Step 9: Stack and Queues [Implementation & Monotonic]

9.1 Foundations & Interconversions

DONE	PROBLEM NAME	DIFFICULTY	CORE MECHANICS
☐	Implement Stack using Queue (and vice versa)	Easy	Amortized operation shifting push/pop
☐	Balanced Parentheses Check	Easy	Stack push openers, pop match closures

9.2 Monotonic Stack Core Questions (Critical for Interviews)

DONE	PROBLEM NAME	DIFFICULTY	MONOTONIC LOGIC
☐	Next Greater Element (I & II)	Medium	Maintain strict decreasing index stack
☐	Trapping Rain Water	Hard	Two-pointer bounded max / Monotonic build
☐	Largest Rectangle in Histogram	Hard	Find Next Smaller Left & Right bounds
☐	Sliding Window Maximum	Hard	Monotonic Deque index optimization

Step 10: Sliding Window & Two Pointer

DONE	PROBLEM NAME	DIFFICULTY	STRATEGY TYPE
☐	Longest Substring Without Repeating Characters	Medium	Dynamic sliding window hash map index shift
☐	Max Consecutive Ones III (At most K zeroes)	Medium	Expand right pointer, shrink left if zero-count > K
☐	Longest Repeating Character Replacement	Medium	Window length - dominant frequency max $\leq$ K
☐	Subarrays with K Different Integers	Hard	Solve exactly via: $\text{Exact}(K) = \text{AtMost}(K) - \text{AtMost}(K-1)$

Step 11: Heaps / Priority Queue

DONE	PROBLEM NAME	DIFFICULTY	HEAP STRUCTURE
☐	Kth Largest / Smallest Element in Array	Easy	Min-Heap of size K bounds tracker
☐	Merge K Sorted Lists	Hard	Min-Heap storing leading nodes of lists
☐	Find Median from Data Stream	Hard	Dual Balance: Max-Heap (Left) + Min-Heap (Right)

Step 12: Greedy Algorithms

DONE	PROBLEM NAME	DIFFICULTY	GREEDY SORTING POLICY
☐	Assign Cookies	Easy	Sort sizes & greeds; greedy two-pointer match
☐	Fractional Knapsack Problem	Medium	Sort descending by value-to-weight ratio
☐	N Meetings in One Room / Activity Selection	Medium	Sort ascending exclusively by task end time
☐	Job Sequencing Problem	Medium	Sort by profit; place in latest empty slot before deadline
☐	Candy Allocation Problem	Hard	Two passes: left-to-right check then right-to-left match

Step 13: Binary Trees [Traversals, Properties, Concept]

13.1 Traversal Fundamentals

DONE	PROBLEM / TRAVERSAL TYPE	DIFFICULTY	STRUCTURAL APPROACH
☐	Preorder, Inorder, Postorder (Recursive & Iterative)	Easy	Stack simulated call-stack sequence
☐	Level Order Traversal (BFS)	Easy	Queue-driven node layer generation
☐	Morris Inorder Traversal	Hard	Threaded binary trees - O(1) space bound

13.2 Core Tree Geometry & Medium Problems

DONE	PROBLEM NAME	DIFFICULTY	RECURSIVE CRITERION
☐	Height of a Binary Tree	Easy	$1 + \max(\text{LeftHeight}, \text{RightHeight})$
☐	Check if Binary Tree is Balanced	Easy	Height delta absolute limit ≤ 1
☐	Diameter of Binary Tree	Medium	Max value of $(\text{LeftHeight} + \text{RightHeight})$ globally
☐	Binary Tree Maximum Path Sum	Hard	Accumulate non-negative branch splits
☐	Lowest Common Ancestor (LCA)	Medium	Split node finder from depth recursion return

Step 14: Binary Search Trees [BST Operations]

DONE	PROBLEM NAME	DIFFICULTY	BST PROPERTY LOGIC
☐	Search in a BST	Easy	Move left if target < root else move right
☐	Insert / Delete a Node in BST	Medium	Isolate node; attach successor/predecessor
☐	Check if a Tree is BST or Not	Medium	Pass tracking range bounds [Min, Max] down
☐	Construct BST from Preorder Traversal	Medium	Use upper-bound validation parameters

Step 15: Graphs [BFS/DFS, Shortest Paths, MST]

15.1 Graph Traversal & Cycle Detection

DONE	PROBLEM NAME	DIFFICULTY	CORE GRAPH MECHANISM
☐	BFS & DFS Traversals	Easy	Queue tracking vs Functional Recursion Stack
☐	Cycle Detection in Undirected Graph	Medium	BFS/DFS track parent mappings for overlaps
☐	Cycle Detection in Directed Graph	Medium	Track active DFS recursive recursion-stack array
☐	Topological Sort (Kahn's Algorithm)	Medium	In-degree array + Queue removal cycles

15.2 Advanced Shortest Paths & MST

DONE	PROBLEM NAME	DIFFICULTY	ALGORITHMIC ENGINE
☐	Dijkstra's Algorithm (Shortest Path)	Medium	Priority Queue min-distance relaxed lookups
☐	Bellman-Ford Algorithm	Hard	Relax edges N-1 times; catch negative weights
☐	Floyd-Warshall Algorithm	Hard	All-pairs multi-stage matrix DP [i][j] via k
☐	Prim's / Kruskal's Algorithm (MST)	Hard	Greedy minimum weight edge edge-cuts / DSU
☐	Disjoint Set Union (DSU) Implementation	Medium	Path Compression + Union by Rank/Size

Step 16: Dynamic Programming [Subsequences, Grids, Games]

16.1 DP Foundations & Matrix Grids

DONE	PROBLEM NAME	DIFFICULTY	DP PARADIGM TRANSITION
☐	Climbing Stairs / Fibonacci Patterns	Easy	1D State Memoization: $dp[i] = dp[i-1] + dp[i-2]$
☐	Frog Jump / House Robber	Medium	Maximize non-adjacent positional subproblems
☐	Grid Unique Paths (I & II)	Medium	2D Grid transitions: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$
☐	Minimum Path Sum in Grid	Medium	Matrix cell edge accumulations boundary evaluation

16.2 Classic Advanced DP Paradigms

DONE	PROBLEM NAME	DIFFICULTY	OPTIMAL RECURRENCE STATE
☐	0/1 Knapsack Problem	Medium	$f(ind, wt) = \max(\text{pick}, \text{non-pick})$ bounds match
☐	Coin Change (I & II)	Medium	Unbounded knapsack combinatorial summation
☐	Longest Common Subsequence (LCS)	Medium	If match: $1+f(i-1,j-1)$ else $\max(f(i-1,j), f(i,j-1))$
☐	Edit Distance	Hard	Min operations track (Insert, Delete, Replace)
☐	Longest Increasing Subsequence (LIS)	Hard	Include/exclude index scanning state or Binary Search

Step 17 & 18: Advanced Tries & Segment Trees

DONE	PROBLEM NAME	DIFFICULTY	ADVANCED MECHANICS
☐	Implement Trie (Insert, Search, StartsWith)	Medium	Prefix tree node structures with character pointers
☐	Maximum XOR of Two Numbers in an Array	Hard	Bitwise Trie tree lookup prioritizing inverse bits
☐	Segment Tree Setup (Range Query & Point Update)	Hard	Divide range bounds by half; full tree array build
☐	Lazy Propagation in Segment Trees	Hard	Amortize structural interval updates down dynamically